# GeoClaw-claude

## Python 地理信息分析工具集

技　术　文　档

版本 v1.0.0 · 2025 年 3 月

UrbanComp Lab — https://urbancomp.net

MIT License · Copyright © 2025 UrbanComp Lab

# 目 录

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 1. 项目概述

## 1.1 背景与目标

GeoClaw-claude 是由 UrbanComp Lab（https://urbancomp.net）开发的轻量级 Python 地理信息分析工具集。项目参考 QGIS Processing Framework 的模块化架构，以 Python 原生生态为基础，面向城市地理空间数据分析场景，提供从数据获取、空间分析到可视化制图的完整工作流。

工具集的名称来源于其核心定位：GeoSpatial Analysis + Claude AI 驱动的 Skill 系统。用户不仅可以使用内置分析算法，还可以通过编写符合规范的 Python Skill 脚本，利用 Anthropic Claude API 实现 AI 辅助的空间数据分析与解读。

## 1.2 主要特性

- **全流程覆盖：**从 OSM/WFS/HTTP 数据下载，到空间分析、路网计算、栅格处理，再到静态制图与交互地图，形成完整闭环。
- **CLI 优先：**提供 geoclaw-claude 命令行工具，支持 onboard 初始化、配置管理、Skill 运行、数据下载，无需编写代码即可完成分析。
- **AI Skill 系统：**用户可编写符合规范的 Python 脚本，通过 SkillContext 接口调用 Claude AI 进行数据解读，支持本地安装与共享。
- **中国数据友好：**内置 WGS84/GCJ-02/BD-09 坐标系互转，支持天地图 POI API，覆盖国内数据使用场景。
- **科研向设计：**所有函数均有详细文档字符串、TODO 标注，方便研究人员理解与二次开发。

## 1.3 适用人群

| 用户类型 | 适用场景 |
|---|---|
| 科研人员 / 博士生 | 城市空间分析、POI 分布研究、路网可达性、栅格地形分析 |
| 实验室本科生 | 学习 GIS 编程、复现论文实验、完成课程设计 |
| 外部合作者 | 快速上手武汉/其他城市数据分析，使用内置 Skill |
| 工程开发者 | 二次开发，自定义 Skill，集成到已有数据处理流程 |

## 1.4 版本历史

| 版本 | 日期 | 主要更新 |
|---|---|---|
| v1.0.0 | 2025-03 | 首个正式版本：CLI、配置系统、Skill 框架、路网分析（最短路径/等时圈/服务区）、栅格分析（DEM/坡度/重分类/分区统计）、坐标转换、内置 Skill、安装脚本 |
| v0.2.0 | 2025-02 | CLI 骨架（click）、Skill 系统原型、远程数据下载框架、路网模块骨架 |
| v0.1.0 | 2025-01 | 初始版本：GeoLayer 核心类、OSM 下载、空间分析、静态制图、武汉示例 |

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 2. 工程结构

## 2.1 目录树

解压或克隆后，项目根目录结构如下：

```
geoclaw_claude_release/
├── geoclaw_claude/           # 主包（Python 源码）
│   ├── __init__.py           # 版本、作者、快速导入
│   ├── config.py             # 配置管理器
│   ├── cli.py                # 命令行工具入口
│   ├── skill_manager.py      # Skill 加载/注册/执行引擎
│   ├── core/
│   │   ├── layer.py          # GeoLayer：核心图层类
│   │   └── project.py        # GeoClawProject：项目管理
│   ├── analysis/
│   │   ├── spatial_ops.py    # 矢量空间分析（缓冲/叠加/KDE）
│   │   ├── network.py        # 路网分析（最短路径/等时圈）
│   │   └── raster_ops.py     # 栅格分析（DEM/坡度/重分类）
│   ├── cartography/
│   │   ├── renderer.py       # 静态/交互式地图渲染
│   │   └── map_composer.py   # 多图层排版与出图
│   ├── io/
│   │   ├── osm.py            # OSM/Overpass 数据下载
│   │   ├── vector.py         # 矢量文件读写
│   │   └── remote.py         # HTTP/WFS/天地图远程下载
│   ├── utils/
│   │   └── coord_transform.py # WGS84/GCJ02/BD09 坐标转换
│   └── skills/
│       └── builtin/          # 内置 Skill 脚本
│           └── hospital_coverage.py
├── data/wuhan/               # 武汉示例数据（7 个 GeoJSON）
├── examples/
│   └── wuhan_analysis.py     # 武汉城市分析完整示例
├── tests/
│   └── test_environment.py   # 环境测试脚本
├── docs/
│   └── GeoClaw-claude_v1.0.0.docx  # 本文档
├── install.sh                # 一键安装脚本
├── setup.py                  # pip 安装配置
├── requirements.txt          # 依赖列表
├── README.md                 # 项目主页
└── CHANGELOG.md              # 版本历史
```

## 2.2 核心数据流

工具集的核心数据流如下图所示（文字描述）：

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
数据来源                    分析处理                    输出
──────────────────────────────────────────────────────────────────

OSM / Overpass API  ──▶  GeoLayer (核心容器)  ──▶  静态地图 (.png)
HTTP / WFS          ──▶  spatial_ops          ──▶  交互地图 (.html)
本地文件            ──▶  network analysis     ──▶  GeoJSON / SHP
天地图 API          ──▶  raster_ops           ──▶  GeoTIFF
                         Skill + AI API       ──▶  分析报告 (.txt)
```

所有中间结果均以 GeoLayer（矢量）或 RasterLayer（栅格）对象封装，保持统一接口。

## 2.3 模块依赖关系

| 模块 | 依赖的外部库 |
|---|---|
| core/layer.py | geopandas, shapely, pyproj |
| analysis/spatial_ops.py | geopandas, shapely, numpy, scipy（KDE） |
| analysis/network.py | osmnx, networkx, geopandas, numpy |
| analysis/raster_ops.py | rasterio, numpy, geopandas |
| cartography/ | matplotlib, folium, contextily（可选）, mapclassify（可选） |
| io/osm.py | requests, geopandas |
| io/remote.py | requests, geopandas |
| cli.py | click |
| skill_manager.py | importlib（标准库）, anthropic（可选，AI 功能） |
| utils/coord_transform.py | math（标准库，无外部依赖） |

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 3. 实现原理

## 3.1 核心数据结构：GeoLayer

GeoLayer 是整个工具集的核心数据容器，封装了 GeoDataFrame 并附加了元数据与辅助方法：

```
class GeoLayer:
    data:     GeoDataFrame   # 底层空间数据
    name:     str            # 图层名称
    source:   str            # 数据来源描述
    crs:      str            # 坐标参考系（EPSG 代码）

    # 核心方法
    def reproject(self, epsg)     # 坐标系转换
    def to_utm(self)              # 自动转最佳 UTM 分带（精确度量）
    def summary(self)             # 打印图层摘要
    def __len__(self)             # 要素数量
```

所有空间分析函数接受 GeoLayer 作为输入，返回新的 GeoLayer（不修改原始数据），保证数据不可变性，方便链式操作。

## 3.2 空间分析原理

### 3.2.1 缓冲区与叠加分析

缓冲区分析（buffer）会自动将图层投影到 UTM 坐标系（单位：米）后执行缓冲，再投影回 WGS84，避免地理坐标系下距离计算失真的问题。叠加分析（intersect/union/clip）均基于 shapely 几何运算实现。

### 3.2.2 KDE 核密度估计

KDE（Kernel Density Estimation）使用 scipy.stats.gaussian_kde，以点要素的 UTM 坐标为输入，在指定分辨率的网格上估算密度值，输出为栅格图像或等值线 GeoLayer。带宽（bandwidth）支持 Scott 法则（默认）或手动指定。

### 3.2.3 最近邻分析

最近邻分析（nearest_neighbor）当前使用 GeoDataFrame 的 sjoin_nearest 方法，注意：该操作在地理坐标系下计算的距离值单位为度，需先转 UTM 才能获得米制距离结果（已标记为 TODO 待修复）。

## 3.3 路网分析原理

### 3.3.1 路网构建

路网数据通过 osmnx 库从 OpenStreetMap 下载，并以 networkx.MultiDiGraph 格式存储。下载后自动调用 add_edge_speeds() 和 add_edge_travel_times() 为每条路段添加速度和行程时间属性，作为后续分析的权重。

### 3.3.2 最短路径

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

最短路径使用 networkx.shortest_path()（Dijkstra 算法），支持 travel_time（秒）和 length（米）两种权重。结果以 LineString GeoLayer 形式返回，携带总距离、行程时间、平均速度等属性。

> 💡 *起终点坐标通过 nearest_node() 映射到路网最近节点，若节点距原始坐标超过 500m 会给出警告。*

### 3.3.3 等时圈（Isochrone）

等时圈算法流程：

- 以中心点的最近路网节点为起点，调用 networkx.single_source_dijkstra_path_length() 获取所有 ≤ T 分钟可达节点
- 取所有可达节点的地理坐标点集
- 计算点集的凸包（convex_hull）作为等时圈多边形
- 投影到 UTM 后做 buffer(300m).buffer(-150m) 平滑处理，减少凸包的锯齿感
- 返回按时间分档的多边形 GeoLayer

> 💡 *当前使用凸包近似，未来计划改用 alpha shape 以更精确反映路网形态（见 TODO）。*

## 3.4 栅格分析原理

### 3.4.1 RasterLayer 数据结构

RasterLayer 封装了 numpy 数组和 rasterio 的 affine 变换矩阵：

```
@dataclass
class RasterLayer:
    data:      np.ndarray    # shape: (bands, rows, cols)
    transform: affine.Affine # 地理变换矩阵
    crs:       CRS           # 坐标参考系
    nodata:    float         # 无效值标记
    name:      str
```

### 3.4.2 坡度计算

坡度（slope）使用 numpy.gradient() 在 x/y 方向分别求偏导，通过 arctan(sqrt(dz/dx² + dz/dy²)) 得到坡度弧度值。针对地理坐标系（单位：度）输入，会自动将像元大小换算为米（经度方向考虑纬度余弦校正）。

### 3.4.3 栅格计算器

raster_calc() 接受 numpy 表达式字符串和关键字参数（变量名→RasterLayer），使用 Python eval() 在隔离命名空间中执行计算，支持所有 numpy 函数。例如：

```
# NDVI = (近红外 - 红波段) / (近红外 + 红波段)
ndvi = raster_calc("(nir - red) / (nir + red)", nir=nir_band, red=red_band)
```

> ⚠️ **注意：**eval() 执行用户输入存在安全风险，当前通过限制 __builtins__ 缓解，生产环境建议增加沙箱。

## 3.5 Skill 系统原理

Skill 系统的核心是动态模块加载机制：

- SkillManager 扫描内置目录（skills/builtin/）和用户目录（~/.geoclaw_claude/skills/）

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License 

- 使用 importlib.util.spec_from_file_location() 动态加载每个 .py 文件为 Python 模块
- 读取模块级变量 SKILL_META 获取元数据，注册到内部字典 _registry
- 执行时构建 SkillContext 对象注入到 skill.run(ctx)，提供统一的输入/输出/AI 接口

SkillContext.ask_ai() 调用 Anthropic Python SDK，发送用户 prompt 和数据上下文，返回模型文本响应，支持 --ai 开关控制是否启用。

## 3.6 坐标转换原理

中国常用三种坐标系：WGS84（GPS 原始）、GCJ-02（国测局加密，高德/腾讯使用）、BD-09（百度加密，在 GCJ-02 基础上再次加密）。

WGS84→GCJ-02 使用国测局公开的正向变换公式（基于克拉索夫斯基椭球体参数）。逆变换（GCJ-02→WGS84）使用差值迭代近似，误差 < 0.00005°（约 5 米）。GCJ-02↔BD-09 使用百度公开的正/反变换公式，精度更高。

> 💡 *境外坐标（bbox 外）不需要偏移，直接原样返回。*

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 4. 安装与配置

## 4.1 系统要求

| 项目 | 要求 |
|---|---|
| 操作系统 | macOS 12+ / Ubuntu 20.04+ / Windows 10+（WSL2 推荐） |
| Python 版本 | 3.9 或更高（推荐 3.11） |
| 内存 | 最低 4 GB，路网下载/分析推荐 8 GB+ |
| 网络 | 首次运行需连接 OSM Overpass API（可配置国内镜像） |

## 4.2 推荐安装方式（一键脚本）

进入项目根目录后执行：

```
# 标准安装（推荐）
bash install.sh

# 开发模式（代码修改即时生效，适合二次开发）
bash install.sh --dev

# 最小安装（跳过 osmnx/rasterio 等大型依赖）
bash install.sh --mini
```

安装脚本会自动检查 Python 版本，安装核心依赖和扩展依赖，最后验证安装是否成功。

## 4.3 pip 手动安装

```
# 创建并激活虚拟环境（推荐）
python3 -m venv .venv
source .venv/bin/activate          # macOS/Linux
.venv\Scripts\activate             # Windows

# 基础安装
pip install .

# 含全部扩展依赖（路网分析/栅格/AI）
pip install .[full]

# 开发模式
pip install -e .[dev]
```

## 4.4 conda 环境安装

```
conda create -n geoclaw python=3.11
conda activate geoclaw
conda install -c conda-forge geopandas rasterio osmnx
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
pip install .[full]
```

> 💡 *conda-forge 提供的 geopandas/rasterio 通常比 pip 安装更稳定，推荐有 conda 环境的用户优先使用。*

## 4.5 初始化配置（onboard）

安装完成后，运行初始化向导完成一次性配置：

```
geoclaw-claude onboard
```

向导共 5 步，引导配置以下内容：

| 步骤 | 配置项 | 说明 |
|---|---|---|
| 1/5 AI 接口 | Anthropic API Key | 用于 Skill 系统 AI 分析功能，可留空（AI 功能不可用） |
| 1/5 AI 接口 | 默认模型 | 默认 claude-sonnet-4-20250514 |
| 2/5 数据目录 | data_dir / cache_dir / output_dir / skill_dir | 各目录会自动创建 |
| 3/5 网络 | HTTP 代理 / Overpass URL | 国内用户可配代理或指定国内镜像 |
| 4/5 制图 | 默认坐标系 / 输出 DPI | 默认 EPSG:4326 / 150 DPI |
| 5/5 日志 | 日志级别 | INFO（默认）/ DEBUG / WARNING |

配置保存于 ~/.geoclaw_claude/config.json，也可用命令行单独修改：

```
geoclaw-claude config set anthropic_api_key  sk-ant-xxxxxxxx
geoclaw-claude config set proxy              http://127.0.0.1:7890
geoclaw-claude config set overpass_url
https://overpass.openstreetmap.ru/api/interpreter
geoclaw-claude config show
```

## 4.6 环境变量覆盖

任意配置项可通过环境变量覆盖（优先级高于配置文件），格式为 GEOCLAW_<KEY>：

```
export GEOCLAW_ANTHROPIC_API_KEY=sk-ant-xxx
export GEOCLAW_PROXY=http://127.0.0.1:7890
export GEOCLAW_ENABLE_CACHE=false
```

## 4.7 验证安装

```
# 运行环境测试
geoclaw-claude test
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License 

```
# 或直接运行测试脚本
python tests/test_environment.py
```

# 5. API 参考

## 5.1 io 模块：数据读写与下载

### 5.1.1 OSM 数据下载 (io/osm.py)

| 函数 | 返回值 | 说明 |
|---|---|---|
| download_pois(bbox, poi_type, max_results) | GeoLayer | 从 Overpass 下载 POI，支持 hospital/park/metro 等类型 |
| download_boundary(place_name) | GeoLayer | 按地名下载行政边界多边形 |
| download_roads(bbox, road_types, max_results) | GeoLayer | 下载道路网络线要素 |
| download_buildings(bbox, max_results) | GeoLayer | 下载建筑物面要素 |

示例：

```
from geoclaw_claude.io.osm import download_pois, download_boundary

boundary  = download_boundary("武汉市")
hospitals = download_pois(boundary.bounds, poi_type="hospital",
max_results=200)
print(f"下载 {len(hospitals)} 所医院")
```

### 5.1.2 矢量文件读写 (io/vector.py)

| 函数 | 返回值 | 说明 |
|---|---|---|
| load_vector(path, name=None) | GeoLayer | 读取 GeoJSON / SHP / GPKG / CSV（含经纬度列） |
| save_vector(layer, path, driver=None) | str | 保存为 GeoJSON / SHP / GPKG |

### 5.1.3 远程数据下载 (io/remote.py)

| 函数 | 返回值 | 说明 |
|---|---|---|
| download_file(url, dest=None, force=False) | str（路径） | HTTP 下载文件，支持缓存 |
| fetch_url_layer(url, name=None) | GeoLayer | 直接从 URL 加载地理数据 |
| fetch_wfs(endpoint, layer_name, bbox=None) | GeoLayer | 从 WFS 服务获取要素 |
| fetch_tianditu_poi(keyword, bbox, api_key) | GeoLayer | 天地图 POI 搜索 |

## 5.2 analysis 模块：空间分析

### 5.2.1 矢量空间分析 (analysis/spatial_ops.py)

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

| 函数 | 返回值 | 说明 |
|---|---|---|
| buffer(layer, distance, unit='meters') | GeoLayer | 缓冲区分析，自动 UTM 投影保证精度 |
| intersect(layer_a, layer_b) | GeoLayer | 相交分析 |
| union(layer_a, layer_b) | GeoLayer | 合并分析 |
| clip(layer, mask) | GeoLayer | 按掩模裁剪 |
| dissolve(layer, by=None) | GeoLayer | 融合要素（可按字段分组） |
| spatial_join(left, right, how='left') | GeoLayer | 空间连接 |
| nearest_neighbor(source, target) | GeoLayer | 最近邻连接 |
| kde(points, resolution=100, bandwidth=None) | ndarray | 核密度估计，返回密度栅格 |

### 5.2.2 路网分析 (analysis/network.py)

| 函数 | 返回值 | 说明 |
|---|---|---|
| build_network(source, network_type='drive') | nx.Graph | 从地名/bbox/GeoLayer 构建路网 |
| build_network_from_layer(roads) | nx.Graph | 从本地道路图层构建（离线） |
| nearest_node(G, lon, lat) | (node_id, dist_m) | 查找最近路网节点 |
| shortest_path(G, origin, dest, weight='travel_time') | GeoLayer | Dijkstra 最短路径 |
| isochrone(G, center, minutes=[5,10,15]) | GeoLayer | 等时圈多边形 |
| service_areas(G, facilities, minutes=10.0) | GeoLayer | 批量设施服务区分析 |
| network_stats(G, area_km2=None) | dict | 路网统计指标 |

### 5.2.3 栅格分析 (analysis/raster_ops.py)

| 函数 / 类 | 返回值 | 说明 |
|---|---|---|
| load_raster(path, band=None) | RasterLayer | 读取 GeoTIFF 等格式 |
| save_raster(raster, path) | str | 保存为 GeoTIFF |
| slope(dem, units='degrees') | RasterLayer | 坡度（度 / 百分比） |
| aspect(dem) | RasterLayer | 坡向（0-360°） |
| hillshade(dem, azimuth=315, altitude=45) | RasterLayer | 山体阴影 |
| reclassify(raster, breaks) | RasterLayer | 按区间重分类 |
| raster_calc(expression, **layers) | RasterLayer | 栅格计算器（numpy 表达式） |
| zonal_stats(raster, zones, stats=[...]) | GeoLayer | 矢量分区统计 |
| clip_raster(raster, mask) | RasterLayer | 按矢量或 bbox 裁剪 |

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

| 函数 / 类 | 返回值 | 说明 |
|---|---|---|
| resample(raster, scale=0.5) | RasterLayer | 重采样 |

## 5.3 utils：工具函数

### 5.3.1 坐标转换 (utils/coord_transform.py)

| 函数 | 返回值 | 说明 |
|---|---|---|
| wgs84_to_gcj02(lon, lat) | (lon, lat) | WGS84 → GCJ-02（高德/腾讯） |
| gcj02_to_wgs84(lon, lat) | (lon, lat) | GCJ-02 → WGS84（近似逆变换） |
| gcj02_to_bd09(lon, lat) | (lon, lat) | GCJ-02 → BD-09（百度） |
| bd09_to_gcj02(lon, lat) | (lon, lat) | BD-09 → GCJ-02 |
| wgs84_to_bd09(lon, lat) | (lon, lat) | WGS84 → BD-09 |
| bd09_to_wgs84(lon, lat) | (lon, lat) | BD-09 → WGS84 |
| transform_layer(layer, from_cs, to_cs) | GeoLayer | 批量转换整个图层 |

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 6. CLI 命令参考

安装完成后，所有功能均可通过 geoclaw-claude 命令行访问：

## 6.1 命令总览

| 命令 | 功能 |
|---|---|
| geoclaw-claude onboard | 交互式初始化向导 |
| geoclaw-claude config show/get/set | 查看和修改配置 |
| geoclaw-claude test | 验证环境是否正常 |
| geoclaw-claude download osm <地名> | 从 OSM 下载 POI 数据 |
| geoclaw-claude download url <URL> | 从 URL 下载地理数据文件 |
| geoclaw-claude skill list | 列出已安装的 Skill |
| geoclaw-claude skill run <名称> [选项] | 运行指定 Skill |
| geoclaw-claude skill new <名称> | 创建 Skill 模板文件 |
| geoclaw-claude skill install <路径> | 安装本地 Skill 文件 |

## 6.2 命令详解

### download osm

```
geoclaw-claude download osm <地名> [选项]

选项:
  --type, -t   POI 类型（hospital/park/metro/university/road）
  --output, -o 输出文件路径（默认: <地名>_<类型>.geojson）
  --max, -n    最大结果数（默认: 500）

示例:
  geoclaw-claude download osm "武汉市" --type hospital
  geoclaw-claude download osm "北京市朝阳区" --type park -n 200 -o
bj_parks.geojson
```

### skill run

```
geoclaw-claude skill run <skill名称> [选项] [额外参数]

选项:
  --data, -d    输入数据文件路径
  --output, -o  输出目录路径
  --ai / --no-ai  是否启用 AI 分析（需配置 API Key）

示例:
  geoclaw-claude skill run hospital_coverage --data hospitals.geojson
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
geoclaw-claude skill run hospital_coverage --data hospitals.geojson --ai
geoclaw-claude skill run my_analysis --data data.geojson --radius_km=5
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 7. Skill 系统详解

## 7.1 什么是 Skill

Skill 是一个符合规范的 Python 脚本文件，代表一个完整的地理分析流程。用户可以在 Skill 中使用 GeoClaw-claude 的所有分析函数，并通过 SkillContext 接口与 Anthropic Claude AI 交互，实现智能化数据解读。

Skill 与普通 Python 脚本的区别在于：通过标准化的 SKILL_META 元数据和 run(ctx) 入口函数，任何 Skill 都可以被 CLI 一键运行，无需关心数据加载、结果保存等样板代码。

## 7.2 Skill 文件规范

```
# my_analysis.py

SKILL_META = {
    "name":        "my_analysis",       # Skill 唯一标识
    "version":     "1.0.0",
    "author":      "your_name",
    "description": "说明这个 Skill 的功能",
    "inputs": [
        {"name": "input",     "type": "GeoLayer", "desc": "输入图层"},
        {"name": "radius_km", "type": "float",    "desc": "分析半径(km)",
"default": 5.0},
    ],
    "outputs": [
        {"name": "result", "type": "GeoLayer", "desc": "分析结果"},
    ],
}

def run(ctx):
    # 1. 读取输入
    layer     = ctx.get_layer("input")
    radius_km = float(ctx.param("radius_km", 5.0))

    # 2. 执行分析
    from geoclaw_claude.analysis.spatial_ops import buffer
    result = buffer(layer, radius_km * 1000)

    # 3. (可选) AI 解读
    ai_text = ctx.ask_ai(
        "请用中文分析这些点位的空间分布特征，给出优化建议。",
        context_data=layer.summary()
    )
    if ai_text:
        print(f"AI 分析: {ai_text}")
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
    # 4. 返回结果（自动保存到 output_dir）
    return ctx.result(result=result)
```

## 7.3 SkillContext 接口

| 方法 / 属性 | 说明 |
|---|---|
| ctx.get_layer(name='input') | 获取已加载图层，name 默认为 'input'（CLI --data 参数加载的图层） |
| ctx.load_layer(path, name=None) | 从文件路径加载额外图层并注册 |
| ctx.param(key, default=None) | 读取运行参数，支持从 --extra_args 解析 --key=value |
| ctx.ask_ai(prompt, context_data=None) | 调用 Claude API，返回文本响应（需 --ai 开关和 API Key） |
| ctx.result(**kwargs) | 注册分析结果，GeoLayer 类型自动保存为 GeoJSON 到 output_dir |
| ctx.output_dir | 输出目录路径（str） |
| ctx.config | 全局配置对象（Config 实例） |

## 7.4 创建与安装 Skill

```
# 1. 生成模板
geoclaw-claude skill new my_analysis
# → 创建 ~/.geoclaw_claude/skills/my_analysis.py

# 2. 编辑模板，实现 run() 函数

# 3. 安装（从任意路径）
geoclaw-claude skill install ./my_analysis.py

# 4. 验证
geoclaw-claude skill list

# 5. 运行
geoclaw-claude skill run my_analysis --data hospitals.geojson --ai
```

## 7.5 内置 Skill：hospital_coverage

hospital_coverage 是预装的示例 Skill，演示完整的分析流程：

- 读取医院点图层
- 按指定半径（默认 3km）生成服务覆盖缓冲区
- 计算覆盖总面积、单院平均覆盖面积
- 若传入边界图层，计算空间覆盖率
- （启用 --ai 时）调用 Claude API 生成医疗可及性分析报告

```
geoclaw-claude skill run hospital_coverage \
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
    --data data/wuhan/hospitals.geojson \
    --ai
```

# 8. Demo：武汉城市空间分析

项目内置武汉市真实 OSM 数据（data/wuhan/），涵盖边界、医院、地铁站、公园、道路、大学、水体共 7 个图层。以下展示从数据加载到分析制图的完整示例流程。

## 8.1 数据说明

| 文件 | 类型 | 要素数 | 说明 |
|---|---|---|---|
| boundary.geojson | Polygon | 1 | 武汉市行政边界（MultiPolygon） |
| hospitals.geojson | Point | 200 | 医院 POI（OSM healthcare=hospital） |
| metro_stations.geojson | Point | 624 | 地铁站点（含各线路属性） |
| universities.geojson | Point | 62 | 高校（OSM amenity=university） |
| parks.geojson | Point/Polygon | 200 | 公园绿地 |
| roads_main.geojson | LineString | 600 | 主干道路网（highway=primary/secondary） |
| water.geojson | Point | 300 | 水体质心点（长江/汉江等） |

## 8.2 Demo 1：医院分布概览

```
from geoclaw_claude import GeoLayer
from geoclaw_claude.io.vector import load_vector
from geoclaw_claude.analysis.spatial_ops import buffer, kde
from geoclaw_claude.cartography.renderer import render_map

# 加载数据
boundary  = load_vector("data/wuhan/boundary.geojson",  name="边界")
hospitals = load_vector("data/wuhan/hospitals.geojson", name="医院")

# 生成 3km 服务缓冲区
coverage = buffer(hospitals, 3000)

# 渲染地图
render_map(
    layers=[boundary, coverage, hospitals],
    title="武汉市医院 3km 服务覆盖范围",
    output_path="output/01_hospital_coverage.png",
)
```

## 8.3 Demo 2：地铁站等时圈

```
from geoclaw_claude.io.vector import load_vector
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
from geoclaw_claude.analysis.network import build_network, isochrone
from geoclaw_claude.cartography.renderer import render_map

# 构建武汉驾车路网（汉口区域）
G = build_network((113.70, 29.97, 115.08, 31.36), network_type="drive")

# 计算汉口火车站 5/10/15 分钟等时圈
hankou_station = (114.2667, 30.6233)  # 经纬度
iso = isochrone(G, center=hankou_station, minutes=[5, 10, 15])

# 加载地铁站作为参照
metro = load_vector("data/wuhan/metro_stations.geojson")

render_map(
    layers=[iso, metro],
    title="汉口火车站驾车等时圈",
    output_path="output/02_isochrone.png",
)
```

## 8.4 Demo 3：坐标系转换

```
from geoclaw_claude.utils.coord_transform import wgs84_to_gcj02,
transform_layer
from geoclaw_claude.io.vector import load_vector, save_vector

# 单点转换（用于核查高德地图坐标是否吻合）
lon_gcj, lat_gcj = wgs84_to_gcj02(114.30, 30.60)
print(f"GCJ-02: {lon_gcj:.6f}, {lat_gcj:.6f}")

# 批量转换整个图层
hospitals_wgs = load_vector("data/wuhan/hospitals.geojson")
hospitals_gcj = transform_layer(hospitals_wgs, "wgs84", "gcj02")
save_vector(hospitals_gcj, "output/hospitals_gcj02.geojson")
```

## 8.5 Demo 4：使用 Skill（含 AI 分析）

```
# 方式一：命令行运行（推荐，最简单）
geoclaw-claude skill run hospital_coverage \
    --data data/wuhan/hospitals.geojson \
    --ai

# 方式二：Python API 调用
from geoclaw_claude.skill_manager import SkillManager

sm = SkillManager()
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
result = sm.run(
    name="hospital_coverage",
    data="data/wuhan/hospitals.geojson",
    output="output/",
    use_ai=True,
)
print(result["report"])
```

## 8.6 运行完整示例

```
python examples/wuhan_analysis.py
```

该脚本会依次执行：城市概览图、医院覆盖分析、地铁-大学空间关系、KDE 密度热图、交互式 HTML 地图，输出保存至 output/ 目录。

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 9. 常见问题 Q&A

## 安装与环境

### Q: 安装时 geopandas 报错 GDAL 版本不匹配?

A: 推荐通过 conda 安装 geopandas，conda-forge 会自动处理 GDAL 依赖：

```
conda install -c conda-forge geopandas
```

如果必须用 pip，确保先安装系统级 GDAL：

```
# Ubuntu/Debian
sudo apt-get install libgdal-dev
pip install GDAL==$(gdal-config --version) geopandas
```

### Q: osmnx 安装失败 / 体积太大，能跳过吗?

A: 可以。使用 --mini 模式安装，路网分析功能（build_network/isochrone 等）将不可用，但矢量分析、栅格分析、制图、Skill 系统均正常工作：

```
bash install.sh --mini
# 或
pip install .  # 不安装 .[full]
```

### Q: Windows 下 install.sh 无法运行?

A: 推荐使用 WSL2（Windows Subsystem for Linux）运行。若不使用 WSL，可直接用 pip 手动安装：

```
pip install .[full]
geoclaw-claude onboard
```

## 数据下载

### Q: OSM 数据下载超时或失败?

A: 默认连接 overpass-api.de，国内可能不稳定。可换用国内镜像：

```
geoclaw-claude config set overpass_url
https://overpass.openstreetmap.ru/api/interpreter
# 或
geoclaw-claude config set proxy http://127.0.0.1:7890
```

### Q: 下载的 OSM 数据坐标是 WGS84 还是 GCJ-02?

A: OSM 数据均为 WGS84 坐标（EPSG:4326）。若需与高德/百度地图叠加，需转换：

```
from geoclaw_claude.utils.coord_transform import transform_layer
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

```
layer_gcj = transform_layer(layer_wgs84, "wgs84", "gcj02")
```

### Q: 如何下载更多 POI 类型?

A: download_pois() 的 poi_type 参数支持 OSM 标准 tag，常用类型：

| poi_type 值 | 对应 OSM Tag | 说明 |
|---|---|---|
| hospital | healthcare=hospital | 医院 |
| park | leisure=park | 公园 |
| metro | railway=station | 地铁/火车站 |
| university | amenity=university | 大学 |
| school | amenity=school | 中小学 |
| supermarket | shop=supermarket | 超市 |
| restaurant | amenity=restaurant | 餐厅 |
| road | highway=primary/ secondary | 主要道路 |

## 空间分析

### Q: nearest_neighbor 返回的距离全是 0?

A: 这是已知 Bug（见 spatial_ops.py TODO）。原因是在地理坐标系（度）下计算距离，数值极小近似为 0。临时解决方案：先将图层转为 UTM 坐标系再做最近邻分析：

```
layer_utm = layer.to_utm()  # 自动转最佳 UTM
result = nearest_neighbor(layer_utm, target_utm)
```

### Q: 等时圈形状不够精确，明显是凸包?

A: 是的，当前等时圈使用可达节点集的凸包近似，对于网格型路网效果尚可，但对于形状不规则的路网会有较大失真。下一版本计划改用 alpha shape 算法。目前可通过增大路网密度（network_type='all'）来改善。

### Q: 栅格分析提示 rasterio 未安装?

A: rasterio 是可选依赖，需单独安装：

```
pip install rasterio>=1.3
# conda 推荐:
conda install -c conda-forge rasterio
```

## Skill 系统

### Q: ask_ai() 返回 '(AI 分析未启用)'?

A: 需要在运行时加上 --ai 参数，并确保已配置 Anthropic API Key：

```
geoclaw-claude config set anthropic_api_key sk-ant-xxxxxxxx
geoclaw-claude skill run my_skill --data input.geojson --ai
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

### Q: 自定义 Skill 中如何加载多个图层?

A: 使用 ctx.load_layer() 加载额外图层：

```
def run(ctx):
    hospitals = ctx.get_layer("input")         # --data 传入的默认图层
    boundary  = ctx.load_layer("boundary.geojson", name="boundary")
    metro     = ctx.load_layer("metro.geojson",    name="metro")
    ...
```

### Q: Skill 能否访问外部 API 或写数据库?

A: 可以，Skill 是标准 Python 脚本，可以 import 任何库，访问网络或数据库。但请注意：当前 Skill 系统未做沙箱隔离，请只运行可信来源的 Skill 文件。

## 制图

### Q: 中文标签显示乱码或方块?

A: 需要在系统中安装支持中文的字体，并在配置中指定：

```
# 查找系统中的 CJK 字体
fc-list :lang=zh

# 在配置中指定字体路径
geoclaw-claude config set font_cjk /usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc
```

也可在代码中临时设置：

```
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Noto Sans CJK JP', 'SimHei', 'Arial Unicode MS']
matplotlib.rcParams['axes.unicode_minus'] = False
```

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 10. 开发者指南

## 10.1 二次开发环境搭建

```
# 1. fork / clone 仓库
git clone https://github.com/whuyao/GeoClaw_Claude.git
cd GeoClaw_Claude

# 2. 安装开发模式（代码修改即时生效）
bash install.sh --dev

# 3. 安装测试工具
pip install pytest black isort
```

## 10.2 贡献新模块

新增分析模块时，请遵循以下规范：

- 文件放置于对应子包（analysis/ / io/ / utils/），并在子包 __init__.py 中导出
- 文件顶部包含：模块路径注释、功能列表、使用示例、TODO 标注、UrbanComp Lab 版权声明
- 所有公开函数需有完整 docstring（Args / Returns / TODO）
- 函数接受 GeoLayer / RasterLayer 作为输入，返回新对象（不修改原数据）
- 在 tests/test_environment.py 中添加对应测试用例
- 在 CHANGELOG.md 中更新版本说明

## 10.3 版本管理规范

版本号遵循语义化版本 (semver)：MAJOR.MINOR.PATCH

- PATCH（如 1.0.1）：Bug 修复、文档更新，不新增功能
- MINOR（如 1.1.0）：新增功能、新增模块，向下兼容
- MAJOR（如 2.0.0）：不兼容的 API 变更

每次版本更新需同步修改：__init__.py 的 __version__、setup.py 的 version、CHANGELOG.md。

## 10.4 已知问题与 Roadmap

| 优先级 | 模块 | 计划功能 |
|---|---|---|
| 🔥 高 | analysis/spatial_ops | 修复 nearest_neighbor() UTM 投影 Bug |
| 🔥 高 | analysis/network | 多目标最短路径（OD 矩阵） |
| 🔥 高 | analysis/network | alpha shape 替代凸包生成等时圈 |
| 中 | analysis/raster_ops | NetCDF 格式支持（气象数据） |
| 中 | io/vector | KML / GML 格式读写 |
| 中 | cartography | contextily 底图集成 |
| 中 | skill_manager | Skill 沙箱执行（安全隔离） |
| 低 | io/postgis.py | PostGIS 数据库连接器 |

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

| 优先级 | 模块 | 计划功能 |
|---|---|---|
| 低 | reporting/report.py | HTML/PDF 自动分析报告生成 |
| 低 | processing/pipeline.py | 批处理 Pipeline API |

Copyright © 2025 UrbanComp Lab (https://urbancomp.net) — MIT License

# 11. 版权与致谢

## 11.1 版权声明

GeoClaw-claude 由 UrbanComp Lab（https://urbancomp.net）开发并维护，以 MIT 许可证开放使用。


```
Copyright (c) 2025 UrbanComp Lab (https://urbancomp.net)

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.
```


## 11.2 引用方式

如在论文或报告中使用本工具集，请注明来源：

```
UrbanComp Lab. (2025). GeoClaw-claude: Python GIS Toolkit for Urban Analysis.
https://urbancomp.net / https://github.com/whuyao/GeoClaw_Claude
```

## 11.3 第三方依赖

| 库 | 许可证 | 用途 |
|---|---|---|
| geopandas | BSD-3 | 核心空间数据处理 |
| shapely | BSD-3 | 几何运算 |
| pyproj | MIT | 坐标投影 |
| osmnx | MIT | OSM 路网下载与分析 |
| networkx | BSD-3 | 图算法（最短路径/等时圈） |
| rasterio | BSD-3 | 栅格数据读写 |
| matplotlib | PSF | 静态地图渲染 |
| folium | MIT | 交互式 Web 地图 |
| click | BSD-3 | CLI 框架 |
| anthropic | MIT | Claude AI API 客户端 |
| requests | Apache-2.0 | HTTP 请求 |

*— 文档结束 —*

UrbanComp Lab · https://urbancomp.net
Copyright © 2025 UrbanComp Lab · MIT License · GeoClaw-claude v1.0.0